

Convenciones. Todos los logaritmos son en base 2.

Indica qué algoritmo de ordenamiento utilizarías y por qué. Puede haber más de una respuesta correcta: estás escribiendo código para que se ejecute en el próximo vehículo explorador de Marte y clasifique los datos recopilados cada noche. (Piensa en cómo realizar la clasificación con memoria y capacidad de cómputo limitadas).

Restricciones del escenario:

- **Memoria limitada.** La RAM se reparte con el software de vuelo: un auxiliar de $\Theta(n)$ compite contra los datos que se van a ordenar.
- **Cómputo limitado.** Procesador endurecido contra radiación y ventana nocturna fija: el costo se dimensiona con la cota, no con una medición en tierra.
- **Sin supervisión.** Se presupuesta el peor caso, porque no hay quién reinicie una corrida que se pasó de la ventana.
- **Fallos por radiación.** El estado vivo se acota antes del lanzamiento, y una pila cuya profundidad depende de los datos no se deja acotar.

Candidatos:

Algoritmo	Mejor	Promedio	Peor	Memoria extra
Mergesort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
Quicksort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\mathcal{O}(\log n)$ de pila
Heapsort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(1)$
Inserción	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$

Tabla 1: Comparaciones y memoria extra sobre claves distintas.

Elección:

- Heapsort es el único que cumple las dos restricciones a la vez: peor caso $\Theta(n \log n)$ y memoria extra $\Theta(1)$. Mergesort da la cota y cobra $\Theta(n)$; quicksort da la memoria y no la cota.
- Su peor caso coincide con el promedio, así que la ventana se dimensiona con un solo número. Quicksort con pivote fijo cae en $\Theta(n^2)$ cuando el lote ya viene ordenado por la clave, que es el caso si la clave es la marca de tiempo; aleatorizar el pivote vuelve improbable ese caso, no lo elimina.
- El *sift-down* es iterativo y trabaja sobre el mismo arreglo: ni pila que dependa de los datos, ni buffer en vuelo.

Cuándo cambia la respuesta:

- Lotes chicos o casi ordenados por la clave de salida: inserción cuesta $\Theta(n + i)$ con i inversiones, y gana mientras $i \in o(n \log n)$, con la misma memoria $\Theta(1)$.
- Hace falta estabilidad, o el lote no cabe en RAM y se ordena por bloques contra la memoria flash: mergesort, que es estable y accede secuencialmente.
- Claves enteras de rango k acotado y conocido: counting sort ordena en $\Theta(n + k)$, y el arreglo de conteo $\Theta(k)$ se dimensiona antes del lanzamiento porque no depende del lote.

Heapsort ($\Theta(n \log n)$ en el peor caso, $\Theta(1)$ de memoria extra)